DBMS Practical Exam – Complete Solutions

✅ PRACTICAL 1 — DDL Commands & Constraints: Student Course Management System

```sql
-- Create Tables
CREATE TABLE Students (
    student_id   INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age          INT CHECK (age >= 18),
    email        VARCHAR(100) UNIQUE
);

CREATE TABLE Courses (
    course_id   INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL
);

CREATE TABLE Enrollments (
    enroll_id  INT PRIMARY KEY,
    student_id INT NOT NULL,
    course_id  INT NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id)  REFERENCES Courses(course_id)
);

-- Insert Records
INSERT INTO Students VALUES
(1, 'Amit Shah',    20, 'amit@email.com'),
(2, 'Priya Patil',  22, 'priya@email.com'),
(3, 'Ravi Kumar',   19, 'ravi@email.com'),
(4, 'Sneha Joshi',  21, 'sneha@email.com'),
(5, 'Neha Gupta',   23, 'neha@email.com');

INSERT INTO Courses VALUES
(101, 'Computer Engineering'),
(102, 'Information Technology'),
(103, 'Electronics'),
(104, 'Mechanical'),
(105, 'Civil');
```

```
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 2, 102),
(3, 3, 101),
(4, 4, 103),
(5, 5, 104);

-- ALTER: Add email column (already added above; shown separately if needed)
ALTER TABLE Students ADD COLUMN phone VARCHAR(15);

-- Rename Table
ALTER TABLE Enrollments RENAME TO StudentEnrollments;

-- Drop unused table (example)
CREATE TABLE TempTable (id INT);
DROP TABLE TempTable;

-- Truncate
TRUNCATE TABLE StudentEnrollments;
```

✅ PRACTICAL 2 — DDL Commands & Constraints: Online Shopping Database

```sql
-- Create Tables
CREATE TABLE Customers (
    customer_id   INT PRIMARY KEY,
    customer_name VARCHAR(50) NOT NULL,
    email         VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Products (
    product_id   INT PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    price        DECIMAL(10,2) CHECK (price > 0)
);

CREATE TABLE Orders (
    order_id    INT PRIMARY KEY,
    customer_id INT NOT NULL,
    product_id  INT NOT NULL,
```

```
    quantity    INT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (product_id)  REFERENCES Products(product_id)
);

-- Insert Records
INSERT INTO Customers VALUES
(1, 'Amit Shah',   'amit@shop.com'),
(2, 'Priya Patil', 'priya@shop.com'),
(3, 'Ravi Kumar',  'ravi@shop.com'),
(4, 'Sneha Joshi', 'sneha@shop.com'),
(5, 'Neha Gupta',  'neha@shop.com');

INSERT INTO Products VALUES
(1, 'Laptop',    55000.00),
(2, 'Mouse',       500.00),
(3, 'Keyboard',   1200.00),
(4, 'Monitor',   15000.00),
(5, 'Headphones',  2500.00);

INSERT INTO Orders VALUES
(1, 1, 1, 1),
(2, 2, 2, 2),
(3, 3, 3, 1),
(4, 4, 4, 1),
(5, 5, 5, 3);

-- ALTER: Add discount column
ALTER TABLE Products ADD COLUMN discount DECIMAL(5,2) DEFAULT 0.00;

-- Rename Table
ALTER TABLE Orders RENAME TO CustomerOrders;

-- Drop Table
CREATE TABLE OldOrders (id INT);
DROP TABLE OldOrders;
```

✅ PRACTICAL 3 — DML Queries: Employee Database Operations

```sql
```

```
-- Create Table
CREATE TABLE Employee (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    dept     VARCHAR(30),
    salary   DECIMAL(10,2),
    city     VARCHAR(30)
);

-- INSERT
INSERT INTO Employee VALUES
(1, 'Amit Shah',    'IT',      55000, 'Pune'),
(2, 'Priya Patil',  'HR',      42000, 'Mumbai'),
(3, 'Ravi Kumar',   'IT',      61000, 'Pune'),
(4, 'Sneha Joshi',  'Finance', 47000, 'Nagpur'),
(5, 'Neha Gupta',   'IT',      58000, 'Pune'),
(6, 'Raj Mehta',    'HR',      39000, 'Mumbai');

-- UPDATE: Increase salary by 10%
UPDATE Employee SET salary = salary * 1.10;

-- DELETE: Remove employee with emp_id = 6
DELETE FROM Employee WHERE emp_id = 6;

-- Arithmetic Operation
SELECT emp_name, salary, salary * 0.20 AS tax FROM Employee;

-- Logical Operators (AND, OR)
SELECT * FROM Employee WHERE dept = 'IT' AND salary > 55000;
SELECT * FROM Employee WHERE city = 'Pune' OR city = 'Mumbai';

-- Pattern Matching (LIKE)
SELECT * FROM Employee WHERE emp_name LIKE 'A%';
SELECT * FROM Employee WHERE city LIKE '%pur%';

-- String Functions
SELECT UPPER(emp_name) AS Name_Upper, LENGTH(emp_name) AS Name_Length FROM Employee;
```

✅ PRACTICAL 4 — DCL & TCL: Banking System with Transactions

```sql
-- Create Table
CREATE TABLE Accounts (
    acc_id  INT PRIMARY KEY,
    name    VARCHAR(50) NOT NULL,
    balance DECIMAL(12,2) NOT NULL
);

-- Insert Records
INSERT INTO Accounts VALUES
(1, 'Amit Shah',   10000.00),
(2, 'Priya Patil', 25000.00),
(3, 'Ravi Kumar',  15000.00),
(4, 'Sneha Joshi', 30000.00),
(5, 'Neha Gupta',  20000.00);

-- TCL: Money Transfer using Transaction
START TRANSACTION;

SAVEPOINT before_transfer;

UPDATE Accounts SET balance = balance - 5000 WHERE acc_id = 1;
UPDATE Accounts SET balance = balance + 5000 WHERE acc_id = 2;

-- If everything is fine:
COMMIT;

-- To undo till savepoint (if needed):
-- ROLLBACK TO before_transfer;

-- DCL: Grant and Revoke (run as admin/root)
CREATE USER 'bank_user'@'localhost' IDENTIFIED BY 'password123';

GRANT SELECT ON Accounts TO 'bank_user'@'localhost';

REVOKE UPDATE ON Accounts FROM 'bank_user'@'localhost';
```

✅ PRACTICAL 5 — Aggregate Functions & Grouping: Sales Analysis

```sql
```

```sql
-- Create Table
CREATE TABLE Sales (
    sale_id    INT PRIMARY KEY,
    product_id INT,
    amount     DECIMAL(10,2),
    region     VARCHAR(30)
);

-- Insert Records
INSERT INTO Sales VALUES
(1, 101, 3000, 'North'),
(2, 102, 7000, 'South'),
(3, 103, 4500, 'North'),
(4, 104, 6000, 'East'),
(5, 105, 2500, 'South'),
(6, 106, 8000, 'East'),
(7, 107, 1500, 'West'),
(8, 108, 5500, 'North');

-- 1. Total sales per region (SUM)
SELECT region, SUM(amount) AS total_sales
FROM Sales
GROUP BY region;

-- 2. Average sales (AVG)
SELECT AVG(amount) AS average_sales FROM Sales;

-- 3. Regions with total sales > 5000 (HAVING)
SELECT region, SUM(amount) AS total_sales
FROM Sales
GROUP BY region
HAVING SUM(amount) > 5000;
```

✅ PRACTICAL 6 — Aggregate Functions & Grouping: Student Performance Report

```sql
-- Create Table
CREATE TABLE Marks (
    mark_id      INT PRIMARY KEY,
    student_name VARCHAR(50),
```

```
    subject     VARCHAR(50),
    marks       INT
);

-- Insert Records
INSERT INTO Marks VALUES
(1, 'Amit',  'Maths',   85),
(2, 'Priya', 'Maths',   72),
(3, 'Ravi',  'Science', 90),
(4, 'Sneha', 'Science', 65),
(5, 'Neha',  'English', 78),
(6, 'Raj',   'English', 55),
(7, 'Amit',  'Science', 80),
(8, 'Priya', 'English', 74);

-- 1. Highest, Lowest, Average marks
SELECT
    MAX(marks) AS highest,
    MIN(marks) AS lowest,
    AVG(marks) AS average
FROM Marks;

-- 2. Count of students per subject
SELECT subject, COUNT(*) AS student_count
FROM Marks
GROUP BY subject;

-- 3. Subjects with average marks > 70 (HAVING)
SELECT subject, AVG(marks) AS avg_marks
FROM Marks
GROUP BY subject
HAVING AVG(marks) > 70;
```

✅ PRACTICAL 7 — JOIN Operations & Views: Library Management

```sql
-- Create Tables
CREATE TABLE Books (
    book_id    INT PRIMARY KEY,
    title      VARCHAR(100) NOT NULL,
```

```
    author     VARCHAR(50)
);

CREATE TABLE Issue (
    issue_id    INT PRIMARY KEY,
    book_id     INT,
    member_name VARCHAR(50),
    issue_date  DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id)
);

-- Insert Records
INSERT INTO Books VALUES
(1, 'DBMS Concepts',     'Korth'),
(2, 'Operating Systems', 'Silberschatz'),
(3, 'Data Structures',   'Tanenbaum'),
(4, 'Computer Networks', 'Forouzan'),
(5, 'Algorithms',        'Cormen');

INSERT INTO Issue VALUES
(1, 1, 'Amit',  '2024-01-10', '2024-01-20'),
(2, 3, 'Priya', '2024-01-12', NULL),
(3, 4, 'Ravi',  '2024-01-15', NULL);

-- 1. INNER JOIN: Show only issued books
SELECT b.book_id, b.title, i.member_name, i.issue_date
FROM Books b
INNER JOIN Issue i ON b.book_id = i.book_id;

-- 2. LEFT JOIN: All books with issue details (NULL if not issued)
SELECT b.book_id, b.title, i.member_name, i.issue_date
FROM Books b
LEFT JOIN Issue i ON b.book_id = i.book_id;

-- 3. RIGHT JOIN: All issued records with book details
SELECT b.title, i.issue_id, i.member_name, i.issue_date
FROM Books b
RIGHT JOIN Issue i ON b.book_id = i.book_id;

-- Create VIEW for currently issued books (return_date IS NULL)
CREATE VIEW CurrentlyIssued AS
SELECT b.title, i.member_name, i.issue_date
```

```
FROM Books b
INNER JOIN Issue i ON b.book_id = i.book_id
WHERE i.return_date IS NULL;

-- Query the view
SELECT * FROM CurrentlyIssued;
```

✅ PRACTICAL 8 — Subqueries: Department-Based Employee Analysis

```sql
-- Create Tables
CREATE TABLE Departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    dept_id  INT,
    salary   DECIMAL(10,2),
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

-- Insert Records
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance'),
(4, 'Marketing');

INSERT INTO Employees VALUES
(1, 'Amit Shah',   1, 62000),
(2, 'Priya Patil', 2, 45000),
(3, 'Ravi Kumar',  1, 70000),
(4, 'Sneha Joshi', 3, 50000),
(5, 'Neha Gupta',  1, 58000),
(6, 'Raj Mehta',   2, 55000),
(7, 'Kiran Das',   4, 48000),
(8, 'Pooja Rao',   3, 52000);
```

```sql
-- 1. Employees in IT department
SELECT emp_name, salary
FROM Employees
WHERE dept_id = (SELECT dept_id FROM Departments WHERE dept_name = 'IT');

-- 2. Employees earning more than average salary
SELECT emp_name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);

-- 3. Employees earning more than the highest-paid employee in HR
SELECT emp_name, salary
FROM Employees
WHERE salary > (
    SELECT MAX(salary)
    FROM Employees
    WHERE dept_id = (SELECT dept_id FROM Departments WHERE dept_name = 'HR')
);
```

✅ PRACTICAL 9 — Subqueries: Students & Courses

```sql
-- Create Tables
CREATE TABLE Courses (
    course_id   INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL
);

CREATE TABLE Students (
    student_id   INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    course_id    INT,
    marks        INT,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- Insert Records
INSERT INTO Courses VALUES
(1, 'Computer Engineering'),
```

```
(2, 'Information Technology'),
(3, 'Electronics'),
(4, 'Mechanical');

INSERT INTO Students VALUES
(1, 'Amit',  1, 85),
(2, 'Priya', 2, 72),
(3, 'Ravi',  1, 90),
(4, 'Sneha', 3, 65),
(5, 'Neha',  1, 78),
(6, 'Raj',   2, 88),
(7, 'Kiran', 4, 70),
(8, 'Pooja', 3, 60);

-- 1. Students enrolled in "Computer Engineering"
SELECT student_name, marks
FROM Students
WHERE course_id = (
    SELECT course_id FROM Courses WHERE course_name = 'Computer Engineering'
);

-- 2. Students who scored more than the average marks
SELECT student_name, marks
FROM Students
WHERE marks > (SELECT AVG(marks) FROM Students);

-- 3. Students enrolled in the same course as "Amit"
SELECT student_name, marks
FROM Students
WHERE course_id = (
    SELECT course_id FROM Students WHERE student_name = 'Amit'
)
AND student_name != 'Amit';
```

🔶 PL/SQL PRACTICALS (10 & 11)

✅ PRACTICAL 10 — Stored Procedure with Cursor: Employee Salary Update

```sql
-- Step 1: Create Table
CREATE TABLE Employee (
    id     INT PRIMARY KEY,
    name   VARCHAR(50),
    salary DECIMAL(10,2)
);

-- Step 2: Insert Records
INSERT INTO Employee VALUES
(1, 'Amit Shah',   55000),
(2, 'Priya Patil', 42000),
(3, 'Ravi Kumar',  61000),
(4, 'Sneha Joshi', 47000),
(5, 'Neha Gupta',  58000);

-- Step 3: Display BEFORE execution
SELECT * FROM Employee;

-- Step 4: Stored Procedure using Cursor (MySQL / PL/SQL syntax)
DELIMITER $$

CREATE PROCEDURE IncreaseSalary()
BEGIN
    DECLARE done      INT DEFAULT 0;
    DECLARE emp_id    INT;
    DECLARE emp_name  VARCHAR(50);
    DECLARE emp_sal   DECIMAL(10,2);

    DECLARE emp_cursor CURSOR FOR
        SELECT id, name, salary FROM Employee;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN emp_cursor;

    read_loop: LOOP
        FETCH emp_cursor INTO emp_id, emp_name, emp_sal;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;
```

```
        UPDATE Employee SET salary = salary + 1000 WHERE id = emp_id;
    END LOOP;

    CLOSE emp_cursor;
END$$

DELIMITER ;

-- Step 5: Call the procedure
CALL IncreaseSalary();

-- Step 6: Display AFTER execution
SELECT * FROM Employee;
```

✅ PRACTICAL 11 — Triggers: Bank Database System

```sql
-- Step 1: Create Table
CREATE TABLE Bank (
    acc_id  INT PRIMARY KEY,
    name    VARCHAR(50) NOT NULL,
    balance DECIMAL(12,2)
);

-- Step 2: Insert Records
INSERT INTO Bank VALUES
(1, 'Amit Shah',   10000),
(2, 'Priya Patil', 25000),
(3, 'Ravi Kumar',  15000),
(4, 'Sneha Joshi', 30000),
(5, 'Neha Gupta',  20000);

-- Step 3: Trigger — Set balance to 0 if NULL (BEFORE INSERT)
DELIMITER $$

CREATE TRIGGER set_default_balance
BEFORE INSERT ON Bank
FOR EACH ROW
BEGIN
    IF NEW.balance IS NULL THEN
```

```
        SET NEW.balance = 0;
    END IF;
END$$

DELIMITER ;

-- Test Trigger 1: Insert without balance
INSERT INTO Bank (acc_id, name) VALUES (6, 'Test User');
SELECT * FROM Bank WHERE acc_id = 6;
-- Output: balance = 0 ✅

-- Step 4: Trigger — Prevent negative balance (BEFORE UPDATE)
DELIMITER $$

CREATE TRIGGER prevent_negative_balance
BEFORE UPDATE ON Bank
FOR EACH ROW
BEGIN
    IF NEW.balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Balance cannot be negative!';
    END IF;
END$$

DELIMITER ;

-- Test Trigger 2: Try to make balance negative
UPDATE Bank SET balance = balance - 50000 WHERE acc_id = 1;
-- Output: Error: Balance cannot be negative! ✅

-- Valid update (should succeed)
UPDATE Bank SET balance = balance - 2000 WHERE acc_id = 1;
SELECT * FROM Bank WHERE acc_id = 1;
```

🍃 MongoDB PRACTICALS (12 & 13)

✅ PRACTICAL 12 — MongoDB CRUD: Student Records

```javascript
// ── Use / Create Database ──────────────────────────────────────────
use studentDB

// ── 1. Insert ONE document ─────────────────────────────────────────
db.students.insertOne({
   name: "Amit Shah",
   age: 21,
   course: "Computer Engineering"
})

// ── 2. Insert MULTIPLE documents ──────────────────────────────────
db.students.insertMany([
   { name: "Priya Patil", age: 20, course: "Information Technology" },
   { name: "Ravi Kumar",  age: 23, course: "Computer Engineering" },
   { name: "Sneha Joshi", age: 19, course: "Electronics" },
   { name: "Neha Gupta",  age: 22, course: "Mechanical" },
   { name: "Raj Mehta",   age: 18, course: "Civil" }
])

// ── 3. Display ALL students ────────────────────────────────────────
db.students.find().pretty()

// ── 4. Find students with age > 21 ─────────────────────────────────
db.students.find({ age: { $gt: 21 } }).pretty()

// ── 5. Update a student's age ──────────────────────────────────────
db.students.updateOne(
   { name: "Sneha Joshi" },
   { $set: { age: 22 } }
)

// ── 6. Delete students with age < 21 ──────────────────────────────
db.students.deleteMany({ age: { $lt: 21 } })

// ── Verify final state ─────────────────────────────────────────────
db.students.find().pretty()
```

✅ PRACTICAL 13 — MongoDB Aggregation: Student Marks

```javascript
// ── Use / Create Database ──────────────────────────────────────────
use studentDB

// ── 1. Insert Records ──────────────────────────────────────────────
db.marks.insertMany([
  { name: "Amit",  marks: 85, subject: "Maths" },
  { name: "Priya", marks: 72, subject: "Maths" },
  { name: "Ravi",  marks: 90, subject: "Science" },
  { name: "Sneha", marks: 65, subject: "Science" },
  { name: "Neha",  marks: 78, subject: "English" },
  { name: "Raj",   marks: 55, subject: "English" },
  { name: "Kiran", marks: 80, subject: "Maths" },
  { name: "Pooja", marks: 74, subject: "Science" }
])

// ── 2a. Average marks per subject ──────────────────────────────────
db.marks.aggregate([
  { $group: {
      _id: "$subject",
      avg_marks: { $avg: "$marks" }
  }}
])

// ── 2b. Count of students per subject ──────────────────────────────
db.marks.aggregate([
  { $group: {
      _id: "$subject",
      student_count: { $sum: 1 }
  }}
])

// ── 2c. Max marks per subject ──────────────────────────────────────
db.marks.aggregate([
  { $group: {
      _id: "$subject",
      max_marks: { $max: "$marks" }
  }}
])

// ── 2d. Total marks per subject ────────────────────────────────────
```

```
db.marks.aggregate([
  { $group: {
      _id: "$subject",
      total_marks: { $sum: "$marks" }
  }}
])

// ── 3. Sort students by marks (Descending) ──────────────────────
db.marks.find().sort({ marks: -1 }).pretty()

// ── Bonus: Combined pipeline — avg, count, max per subject ──────
db.marks.aggregate([
  { $group: {
      _id: "$subject",
      avg_marks:     { $avg: "$marks" },
      total_marks:   { $sum: "$marks" },
      max_marks:     { $max: "$marks" },
      student_count: { $sum: 1 }
  }},
  { $sort: { avg_marks: -1 } }
])
```

> **Note:**
> - SQL Practicals (1–9): Compatible with **MySQL 8.x**
> - PL/SQL Practicals (10–11): Uses **MySQL Stored Procedures & Triggers** syntax (`DELIMITER $$`)
> - MongoDB Practicals (12–13): Run in **MongoDB Shell (mongosh)** or **MongoDB Compass**